Practical Improvements on BKZ Algorithm

Ziyu Zhao(✉) and Jintai Ding(✉)

Yau Mathematical Center, Tsinghua University, Beijing, China
ziyuzhao0008@outlook.com, jintai.ding@gmail.com

Abstract. Lattice problems such as NTRU and LWE problems are widely used as the security base of post-quantum cryptosystems. And currently, lattice reduction by BKZ algorithm is the most efficient way to solve them. In this paper, we give four further improvements on BKZ algorithm, which can be used for SVP subroutines based on enumeration and sieving. These improvements in combination provide a speed-up of $2^{3\text{-}4}$ in total. So all the lattice-based NIST PQC candidates lose 3–4 bits of security in concrete attacks. Using these new techniques, we solved the 656 and 700 dimensional ideal lattice challenges in 380 and 1787 thread hours, respectively. The cost of the first one (also used an enumeration-based SVP subroutine) is much less than the previous records (4600 thread hours). One can still simulate the improved BKZ algorithm to find the blocksize strategy that makes Pot of the basis (defined in Sect. 4.2) decrease as fast as possible, which means the length of the first basis vector decrease the fastest if we accept the GSA assumption. It is useful for analyzing concrete attacks on lattice-based cryptography.

Keywords: Lattice-based cryptography · Lattice reduction · BKZ algorithm

1 Introduction

Lattice-based cryptography is nowadays an important part of post-quantum cryptography. The security of it is mainly based on some lattice problems, such as learning with error problem (LWE) [9,17,25] and NTRU problem [13]. These problems can be reduced to approximate shortest vector problem (ASVP) [5,16,19,20], i.e. to find a relatively short vector given a lattice basis. And now lattice attacks are the most efficient way to solve such problems, thus it is important to know the concrete hardness of ASVP.

Currently, given a lattice basis, there are two types of algorithms to find short vectors in the lattice. One is SVP algorithms like enumeration [6,7,14,24,27,31] and sieving [3,10,18,22], which can find almost the shortest vector in the lattice but the cost is at least exponential in the dimension of the lattice. These SVP algorithms can only be applied to lattices with a small dimension when there are restricted resources. Another type of algorithm, for instance,

S. Dolev et al. (Eds.): CSCML 2023, LNCS 13914, pp. 273–284, 2023.
https://doi.org/10.1007/978-3-031-34671-2_19

LLL algorithm [15,21] and BKZ [28,31] algorithm can work on high dimensional lattices in a realistic time. LLL algorithm is extremely fast and often used as preprocessing, BKZ algorithm gives a bridge from the shortest vector in small dimension to short vectors with the same root Hermite factor in high dimension.

BKZ algorithm was first proposed by Schnorr in the 80's. It does enumeration on local blocks to find short vectors and then inserts the new vectors in the basis. Larger local blocksize gives shorter vector and takes more time. In 2011, Chen-Nguyen used a pruning technique in the enumeration step, it makes BKZ algorithm with a higher local blocksize practicable [4]. In 2016, Yuntao Wang et al. proposed their improved progressive BKZ [2], they got an optimized blocksize strategy based on their simulation of the total enumeration cost. It starts with a small blocksize, and increases the blocksize in a well-organized manner, making the algorithm significantly faster.

In this paper, we will give four further improvements on BKZ algorithm. We replace the insertion in BKZ by processing the local projected lattice. Then we can use a jumping technique to move more than one step each time in the BKZ tours. In the last several tours of BKZ, we only run the SVP subroutines on the first several indexes, which saves half of the time of these tours, then we give an end of the reduction by running a much heavier SVP subroutine.

We applied these techniques in lattice reduction with SVP subroutines based on both enumeration and sieving. We implemented the new BKZ algorithm and tested it on ideal lattice challenges (the ideal lattice structure is never used). The running result shows we get a speed up with a factor 2^{3-4}, which may be further improved since we did not use a well-organized progressive BKZ (for the enumeration-based BKZ, our blocksize are simply $80, 88, 96, \cdots$). Moreover, our new BKZ algorithm is still easy to simulate (as BKZ 2.0 [4]) if we know the behavior of the SVP subroutine well.

Road Map. In Sect. 2, we present some basic facts about lattice and introduce the notations. Then in Sect. 3, we will recall the developments of BKZ algorithm in history. Our further improvements on BKZ will be given in Sect. 4. The information about the lattice challenge results is in Sect. 5.

2 Preliminaries

Lattice is discrete subgroup in $\mathbb{R}^m$. A lattice L always admits an integral basis $\mathbf{B} = \{\mathbf{b}_1, \mathbf{b}_2, \cdots, \mathbf{b}_n\}$ such that each vector $\mathbf{v}$ in L can be represented uniquely as an integral linear combination of $\mathbf{B}$, i.e. $\mathbf{v} = \lambda_1\mathbf{b}_1 + \cdots + \lambda_n\mathbf{b}_n$, $\lambda_i \in \mathbb{Z}$. We say n is the dimension of the lattice. The determinant of the lattice $\det(L)$ is defined to be $\sqrt{\det(\mathbf{B}\mathbf{B}^T)}$. It is equal to the absolute value of $\det(\mathbf{B})$ if $m = n$.

Gram-Schmidt Orthogonalization. The Gram-Schmidt orthogonalization of $\mathbf{B}$ is given by $\mathbf{B}^* = (\mathbf{b}_1^*, \cdots, \mathbf{b}_n^*)$ where $\mathbf{b}_i^*$ is defined by

$$\mathbf{b}_i^* = \mathbf{b}_i - \sum_{j=1}^{i-1} \mu_{ij}\mathbf{b}_j^*, \ \mu_{ij} = \frac{\langle \mathbf{b}_i, \mathbf{b}_j^* \rangle}{\|\mathbf{b}_j^*\|^2}$$

We further denote by B_i the square of $\|\mathbf{b}_i^*\|$, we will call $[B_1, B_2, \cdots, B_n]$ the distance vector of the basis $\mathbf{B}$. The distance vector of a basis contains lots of information about it and this notation will be heavily used in the analysis of BKZ algorithm. We should also introduce the concept of local projected lattice. Let π_i be the orthogonal projection to $\text{span}(\mathbf{b}_1, \cdots, \mathbf{b}_{i-1})^\perp$. Then we define the local projected lattice $L_{[i,j]}$ to be the lattice spanned by $B_{[i,j]} = (\pi_i(\mathbf{b}_i), \cdots, \pi_i(\mathbf{b}_j))$.

Gaussian Heuristic. Because of the discreteness, the shortest nonzero vector in L exists (not unique in general). It is extremely hard to compute the shortest vector (proved to be NP-hard), but the length of it is estimated to be

$$\text{GH}(L) = \frac{\Gamma(\frac{n}{2}+1)^{\frac{1}{n}}}{\sqrt{\pi}} \cdot \det(L)^{\frac{1}{n}} \approx \sqrt{\frac{n}{2\pi e}} \cdot \det(L)^{\frac{1}{n}}$$

when the lattice is "*random*" and n is not too small. In practice it works well if $n \geqslant 40$.

Root Hermite Factor. For a vector $\mathbf{v}$ in a n dimensional lattice L, we define the root Hermite factor to be

$$\delta = \text{RHF}(\mathbf{v}) = \left(\frac{\|\mathbf{v}\|}{\det(L)^{\frac{1}{n}}}\right)^{\frac{1}{n}}$$

as in [8], the root Hermite factor measures the quality of the vector. The hardness to get a vector of a fixed length mainly depends on its root Hermite factor.

HKZ Reduced Basis. We say a lattice basis $\mathbf{B}$ of an n dimensional lattice L is HKZ-reduced if for each $1 \leqslant i \leqslant n$, the first vector of $B_{[i,n]}$ is the shortest nonzero vector of $L_{[i,n]}$.

3 History of BKZ Algorithm

3.1 The Original Algorithm

The first version of BKZ algorithm was proposed by Schnorr and Euchner as a generalization of the LLL algorithm [31]. Briefly, LLL algorithm gives the basis an order, then always reduces the latter vector by the former ones, and after the reduction is done, it tries to make the former one shorter (in the corresponding local projected lattice) by swapping contiguous vector pairs. The algorithm terminates when no more swaps or reductions can be done. The first vector of the output basis is of length about 1.02^n times the Gaussian heuristic in practice (see [8]), and the running time is polynomial in n, where n is the dimension of the lattice.

BKZ algorithm replaces the *swap* in LLL algorithm by a full enumeration in the local projected lattice to get a shorter (also in the corresponding local

projected lattice) vector. This vector will be inserted into the basis at a preselected place, and then we use an LLL algorithm to remove the linear dependency. The size of the local projected lattice is fixed and the place to do enumeration is pre-specified. Similar to LLL algorithm, BKZ algorithm terminates when no nontrivial insertion can be done. The algorithm works as follows:

Algorithm 1: BKZ algorithm

Input: a basis $\mathbf{B} = (\mathbf{b}_1, \cdots, \mathbf{b}_n)$, blocksize d and $\delta < 1$
Output: A BKZ-d reduced basis

LLL($\mathbf{B}, \delta$);
while *last epoch did a nontrivial insertion* **do**
  for $i = 1, 2, \cdots, n-1$ **do**
    $h = \max(i + d - 1, n)$;
    $\mathbf{v} =$ full_enum($L_{[i,h]}$); //find the shortest nonzero vector in $L_{[i,h]}$
    if $\|\mathbf{v}\| < \delta \|\mathbf{b}_i^*\|$ **then**
      let $\tilde{\mathbf{v}} = \lambda_i \mathbf{b}_i + \cdots + \lambda_h \mathbf{b}_h$ if $\mathbf{v} = \lambda_i \pi_i(\mathbf{b}_i) + \cdots + \lambda_h \pi_i(\mathbf{b}_h)$;
      LLL($\mathbf{b}_1, \cdots, \mathbf{b}_{i-1}, \tilde{\mathbf{v}}, \mathbf{b}_i, \cdots, \mathbf{b}_{\max(h+1,n)}, \delta$);
      remove the zero vector in the first place.
    else
      LLL($\mathbf{b}_1, \cdots, \mathbf{b}_{\max(h+1,n)}, \delta$);

The running time of BKZ algorithm increases as the blocksize increases. It is not proved to be polynomial in n (the dimension) for fixed blocksize. But for small blocksize d (for example $d < 20$), the algorithm always terminates in a reasonable time and the output quality is significantly improved. For $d = 20$ and n sufficiently large, the length of the shortest nonzero vector it founds is around 1.0128^n times the Gaussian heuristic (see [8]).

3.2 BKZ2.0

In 2011, Chen-Nguyen gave several improvements on the original BKZ algorithm [4], which made BKZ algorithm with a high blocksize (d–100) practicable. The root Hermite factor of the output vector is improved to about 1.0095. They mainly did the following:

The first point is that, for a large blocksize d ($d \geqslant 40$), before no new insertion can be done, there is a long time that the quality of the basis improves poorly [11]. So they used the *early abort* technique to stop the algorithm as soon as the quality of the basis does not improve further. This provides an exponential speed-up in practice [8] without degenerating the output quality.

They also made some modifications to the enumeration step. For a larger blocksize d, experiment shows the running time of BKZ is dominated by the enumeration subroutines. They used pruned enumeration instead of full enumeration. Proper pruning can give an exponential speed up (about $2^{d/4}$) while

the algorithm still outputs the shortest vector with a high probability. They further gave an *extreme pruning* technique [7], which repeats a further pruned enumeration several times. This leads to a speed-up of $2^{d/2}$.

Another thing they did is to preprocess the basis before doing the enumeration. Since the nodes to enumerate will be fewer if the basis has a better quality. Taking some time to do a light reduction will largely reduce the total enumeration time. In BKZ 2.0 they chose a BKZ algorithm with a small blocksize as preprocessing.

3.3 Progressive BKZ

Progressive BKZ mainly means to progressively enlarge the blocksize while doing reductions. The key idea is if an enumeration with a low dimension can further reduce the lattice, there is no need to use a much larger dimension since the cost of SVP is at least exponential in the dimension. This technique was mentioned in several studies including [4,12,30]. These works mainly differ in the way they increase the blocksize. In 2016, [2] did a precise cost estimation of the progressive BKZ, and gave an optimized blocksize strategy. In their estimation, to do a BKZ-100 in an 800-dimensional lattice, their progressive BKZ is $2^{2.7}$ times faster than BKZ 2.0. And it's estimated to be 50 times faster than BKZ 2.0 for solving SVP challenges (see [26]) up to dimension 160.

4 Several Improvements About BKZ Algorithm

In this section, we will give several techniques to further accelerate the BKZ algorithm. These techniques can be used for BKZ based on both sieving and enumeration SVP subroutines. And one can use it almost for free (except for the large final run for sieving, which requires more memory) to get a considerable acceleration in practice.

4.1 Local Basis Processing Instead of Insertion

Currently, we have two types of SVP algorithms, enumeration and sieving. Sieving is faster but requires a large space that grows exponentially in the sieving dimension. To do a single large sieving or enumeration with the hope of finding the shortest vector is generally not the best choice.

The number of nodes on the enumeration tree grows as the basis gets worse [7]. In practice, it's better to do some preprocessing as in BKZ2.0. So the whole enumeration process not only gives a short vector but also a rather good basis. For sieving, one often uses the left progressive sieve to accelerate, whose speed also relies on the quality of the basis. And one needs the first several entries in the distance vector to be small to get a large dimension for free, which saves both time and memory (for sieving techniques, see [1]). Thus it will not lead to much further cost to get a good basis.

Only inserting one short vector like the original BKZ algorithm or BKZ2.0 will waste the almost *free* basis. It's generally better to compute the transform matrix of local processing (on the local projected lattice) and apply it on the vectors of the original basis (succeed by a size reduction). Then the next local basis to apply the SVP algorithms is already only a little bit worse than an HKZ-reduced basis. An obvious gain is we need no more preprocessing for it. This *local basis processing* technique is folklore for sieving-based BKZ, but it is necessary to point it out because it's the foundation of the next technique, and we first used it on enumeration-based BKZ.

Similar to BKZ2.0 [4], we can still simulate the improved BKZ algorithm by simulating the distance vector $(\|\mathbf{b}_1^*\|^2, \cdots, \|\mathbf{b}_n^*\|^2)$ of the lattice. So we can efficiently find the optimized blocksize strategies and predict the precise cost of the reduction.

4.2 Jump by Two or More

After we do a local basis processing with blocksize d, the first vector will be short, and it's easy to see that the next few vectors are not too long also. If we make our working context jump to the right by two indices or more (i.e. to work on $L_{[i+s,j+s]}$ after $L_{[i,j]}$), say we jump s steps after each SVP subroutine, we accelerate by factor s while not degenerating the quality much.

Here a crucial point is to use a blocksize slightly larger than we require. For instance, if we want to do a BKZ with blocksize d, we can choose a $d' = d+s-1$ and every time we jump s steps. The result will not be worse (actually better) than after a tour of BKZ-d without the jump technique, if the output basis of the SVP subroutine has similar quality as an *HKZ-reduced* basis. After the modification, the number of SVP subroutines is only $\frac{1}{s}$ as before, and for each subroutine, the cost is $2^{0.386(s-1)}$ (practically, when d–90) as before if we use SVP algorithm based on 3-sieve. Take $s = 4$ we get a speed-up of at least $2^{0.84}$.

To verify the effectiveness of this method, we generate a 400-dimensional random lattice with determinant 32768^{400}. Then we execute a BKZ reduction to make the first basis vector has length $32768 \cdot 1.01046^{400}$, corresponding to blocksize 75. We use *Pump* in [1] as the SVP-subroutine (the expected dimension for free is set to be 12, so it behaves like HKZ-reduction), run one tour of BKZ with different maximal sieving dimension (MSD) and jumping steps. To measure the quality of a basis, we introduce the following notation:

$$\mathrm{Pot}(L) = \prod_{i=1}^{n} B_i^{n+1-i}$$

For a given lattice, Pot is an increasing function in the root Hermite factor of the first basis vector, if we accept the GSA assumption [29]. So a better basis will have a smaller Pot. We present $\Delta \log_2 \mathrm{Pot}$ and the cost in Table 1 and 2. From the tables, we see simply a larger jumping step (4–6) will lead to a speed-up of $2^{1.2}$. The gain of this technique varies with different SVP subroutines, and in practice is typically 2^1–2^2 (Tables 1, 2).

Table 1. Jumping step = 1

MSD	68	69	70	71	72	73	74	75
cpu hours	2.30	2.74	3.27	3.88	4.69	5.69	6.93	8.52
$\Delta \log_2 \text{Pot}$	463	758	1166	1400	1910	2254	2674	2949
$\frac{\Delta \log_2 \text{Pot}}{\text{cost}}$	201	277	357	361	407	396	385	346

Table 2. Different jumping steps

(MSD, jumping step)	(72, 1)	(73, 2)	(74, 3)	(75, 4)	(76, 5)	(77, 6)	(78, 7)
cpu hours	4.69	2.84	2.31	2.13	2.20	2.30	2.51
$\Delta \log_2 \text{Pot}$	1910	1797	1787	1962	2059	2084	2241
$\frac{\Delta \log_2 \text{Pot}}{\text{cost}}$	407	633	773	920	930	906	858

We notice that [1] also tried a technique called *PumpNJump*. They set the jumping step to be 3, and the gain is about $2^{0.2}$ when the blocksize is high (estimated from Fig. 4 in their paper). We guess it's because they did not use a large enough blocksize.

4.3 Reduce only When We Need

In practice, we usually don't need the whole BKZ reduced basis. What we want is just a short vector (in lattice challenges) or make the tail of the distance vector large enough such that we can get the key hidden in the lattice by a size reduction (in real attacks of lattice-based cryptography). We only introduce the case for lattice challenges here, because the other case is similar.

For example, if we want to do BKZ-d on an n-dimensional lattice. For the last $\lceil \frac{n}{d} \rceil$ tours of the algorithm, we don't need to visit all indexes. In fact, we work as Algorithm 2 instead of Algorithm 3.

Algorithm 2: The last several tours of our BKZ

Input: an n-dimensional lattice L, blocksize d and an SVP algorithm
Output: a reduced basis, denote by L_2

$m = \lceil \frac{n}{d} \rceil$;
for $k = 1, 2, \cdots, m$ **do**
  //do a BKZ tour on $L_{[1, n-kd+1]}$
  for $i = 1, 2, \cdots, n - kd + 1$ **do**
    reduce $L_{[i, i+d-1]}$ by the SVP algorithm;
return L

Algorithm 3: The original BKZ

Input: an n-dimensional lattice L, blocksize d and an SVP algorithm
Output: a reduced basis, denote by L_1

```
m = [n/d];
for k = 1, 2, ··· , m do
    for i = 1, 2, ··· , n − d + 1 do
        reduce L_[i,i+d−1] by the SVP algorithm;
return L
```

Because the BKZ tour on $L_{[1,n-kd+1]}$ only relates to the first $n-(k-1)d$ vectors, the first $n-\left[\frac{n}{d}\right]d+1$ vectors of L_1 and L_2 are the same. But we save at least half the time for these final BKZ tours.

Notice that if we use a progressively larger blocksize, even if we jump by two or more usually we stay on one dimension for only 1–2 tours. This technique at least saves a constant ratio of time for the total challenge since the best SVP algorithm takes exponential time. We present the details of our 700 dimensional lattice challenge below. From Table 3 we see this technique saves about 900 CPU hours for our 700 dimensional challenge, which costs 1787 CPU hours in total (Table 3).

Table 3. Details for the end of our 700 dimensional challenge

MSD	working on	CPU time	a full tour time	min length
91	$L_{[1,578]}$	196.3 h	206.4 h	812896
92	$L_{[1,466]}$	201.5 h	260.2 h	805991
92	$L_{[1,354]}$	152.3 h	258.9 h	787811
93	$L_{[1,242]}$	146.0 h	365.2 h	755466
94	$L_{[1,130]}$	147.0 h	651.9 h	729162

4.4 A Large Final Run

In BKZ for high dimensional lattice, we can choose a much larger dimension d in the last SVP subroutine (working on $[1,d]$) to get a much shorter vector, saving the time for several tours of SVP subroutines with a normal blocksize. Since one tour costs $n/s \cdot T_{svp}$, which is much larger than an SVP subroutine, this method works well in practice (if we are searching for the secret key hidden in the lattice, just do a large enumeration or sieving at the tail of the basis). For our 700-dimensional lattice challenge, we ran a sieving-based SVP algorithm on the first 124 vectors, and got a vector of length 659874. In this case, we saved time for about 2 heaviest tours of BKZ.

5 The Lattice Challenges

The ideal lattice challenge [23] was started in 2012. It provides many different ideal lattices with dimensions up to 1024. The original goal of this challenge is to test the algorithms for finding short vectors in ideal lattices. But we will treat it as high dimensional random lattices to test our BKZ techniques. For each given lattice, a vector shorter than $1.05 \cdot \mathrm{GH}(L)$ can enter the SVP Hall of Fame, and a vector shorter than $n \cdot \det(L)^{\frac{1}{n}}$ can enter the Approximate SVP Hall of Fame. We solved the following challenges (Table 4):

Table 4. The ideal lattice challenges we solved

dim	length	root Hermite factor	total cost	SVP subroutines
656	670275	1.00993	380 thread hours	Enumeration
700	659874	1.00928	1787 thread hours	Sieving

5.1 The Challenge Based on Enumeration

The 656-dimensional challenge was first finished in the summer of 2021, we used a laptop with Intel Core i7-7500U CPU (2.70 GHz) and a non-optimized c++ program which was modified several times while running. The total cost is about 700 core hours (the information uploaded to the website of ideal lattice challenge is wrong). Later we optimized the program and ran it on an Intel Xeon Silver 4208 CPU (2.10 GHz) again. It takes only 380 core hours, much faster than the previous record which takes 4637 thread hours to solve a 652-dimensional approximate SVP challenge.

To get a reduced basis, we used a variant of DeepBKZ [32] as the SVP subroutine. DeepBKZ replaces the LLL in the original BKZ algorithm with DeepLLL (see [31]), which allows *deep insertion*. We modified the algorithm by further checking all short vectors from an enumeration if they can be inserted into some former place, and we always choose the candidate that can be inserted the deepest. The jumping step was 3–4. And we did not use a highly optimized blocksize strategy which may give a further speed up (the blocksize we used was simply 80, 88, 96, $\cdots$).

5.2 The Challenge Based on Sieving

The 700-dimensional challenge was finished recently. We started with a BKZ-80 reduced basis (takes about 200 thread hours), and then used BKZ based on 3-sieve. The Sieving step takes 1587 thread hours, also on the Xeon Silver 4208 CPU (2.10 GHz).

The jumping step was 6. For the maximal sieving dimension, before each tour of BKZ we calculate the current Pot and compute the corresponding blocksize d by the GSA assumption. We then choose the maximal sieving dimension to be d. Such a choice performs well in practice. The expected dimension for free is set to be 18. so we work on a local projected lattice with dimension $d + 18$.

6 Conclusion

For the lattice-based NIST PQC candidates, most of our techniques are practicable and reduce $3 \sim 4$ bits of security in concrete attacks. If we also consider the memory overhead, *a large final run* may not be a good choice since it uses significantly more memory. However, in this paper, our sieving cost is based on the experiments ($2^{0.386d}$ for dim –90). For large sieving dimensions, this cost should be $2^{0.292d}$ if we use the state-of-art sieving algorithm [3]. So for NIST candidates, the speed-up ratio of the jumping technique will be larger than our low dimensional case. A precise estimation of the impact of these techniques on the NIST candidates may be a direction of future work.

References

1. Albrecht, M.R., Ducas, L., Herold, G., Kirshanova, E., Postlethwaite, E.W., Stevens, M.: The general Sieve Kernel and new records in lattice reduction. In: Ishai, Y., Rijmen, V. (eds.) EUROCRYPT 2019. LNCS, vol. 11477, pp. 717–746. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-17656-3_25
2. Aono, Y., Wang, Y., Hayashi, T., Takagi, T.: Improved progressive BKZ algorithms and their precise cost estimation by sharp simulator. In: Fischlin, M., Coron, J.-S. (eds.) EUROCRYPT 2016. LNCS, vol. 9665, pp. 789–819. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-662-49890-3_30
3. Becker, A., Ducas, L., Gama, N., Laarhoven, T.: New directions in nearest neighbor searching with applications to lattice sieving. In: Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms, pp. 10–24. SODA 2016 (2016). https://doi.org/10.1137/1.9781611974331.ch2
4. Chen, Y., Nguyen, P.Q.: BKZ 2.0: better lattice security estimates. In: Lee, D.H., Wang, X. (eds.) ASIACRYPT 2011. LNCS, vol. 7073, pp. 1–20. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-25385-0_1
5. Coppersmith, D., Shamir, A.: Lattice attacks on NTRU. In: Fumy, W. (ed.) EUROCRYPT 1997. LNCS, vol. 1233, pp. 52–61. Springer, Heidelberg (1997). https://doi.org/10.1007/3-540-69053-0_5
6. Fincke, U., Pohst, M.E.: Improved methods for calculating vectors of short length in a lattice. Math. Comput. **44**, 463–471 (1985). https://doi.org/10.1090/S0025-5718-1985-0777278-8
7. Gama, N., Nguyen, P.Q., Regev, O.: Lattice enumeration using extreme pruning. In: Gilbert, H. (ed.) EUROCRYPT 2010. LNCS, vol. 6110, pp. 257–278. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-13190-5_13
8. Gama, N., Nguyen, P.Q.: Predicting lattice reduction. In: Smart, N. (ed.) EUROCRYPT 2008. LNCS, vol. 4965, pp. 31–51. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-78967-3_3
9. Gentry, C., Peikert, C., Vaikuntanathan, V.: Trapdoors for hard lattices and new cryptographic constructions. In: Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing. STOC 2008, Association for Computing Machinery (2008). https://doi.org/10.1145/1374376.1374407
10. Hanrot, G., Pujol, X., Stehlé, D.: Algorithms for the shortest and closest lattice vector problems. In: Chee, Y.M., et al. (eds.) IWCC 2011. LNCS, vol. 6639, pp. 159–190. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-20901-7_10

11. Hanrot, G., Pujol, X., Stehlé, D.: Analyzing blockwise lattice algorithms using dynamical systems. In: Rogaway, P. (ed.) CRYPTO 2011. LNCS, vol. 6841, pp. 447–464. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-22792-9_25
12. Haque, M.M., Rahman, M.O.: Analyzing progressive-BKZ lattice reduction algorithm. Int. J. Comput. Netw. Inf. Secur. **11**, 40–46 (2019)
13. Hoffstein, J., Pipher, J., Silverman, J.H.: NTRU: a ring-based public key cryptosystem. In: Buhler, J.P. (ed.) ANTS 1998. LNCS, vol. 1423, pp. 267–288. Springer, Heidelberg (1998). https://doi.org/10.1007/BFb0054868
14. Kannan, R.: Improved algorithms for integer programming and related lattice problems. Proceedings of the Fifteenth Annual ACM Symposium on Theory of Computing (1983). https://doi.org/10.1145/800061.808749
15. Lenstra, A.K., Lenstra, H.W., Lovász, L.M.: Factoring polynomials with rational coefficients. Math. Ann. **261**, 515–534 (1982)
16. Lindner, R., Peikert, C.: Better key sizes (and attacks) for LWE-based encryption. In: Kiayias, A. (ed.) CT-RSA 2011. LNCS, vol. 6558, pp. 319–339. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-19074-2_21
17. Micciancio, D., Regev, O.: Lattice-based Cryptography. In: In: Bernstein, D.J., Buchmann, J., Dahmen, E. (eds.) Post-Quantum Cryptography, pp. 147–191. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-540-88702-7_5
18. Micciancio, D., Voulgaris, P.: A deterministic single exponential time algorithm for most lattice problems based on voronoi cell computations. Electron. Colloquium Comput. Complex. **17**, 14 (2010). https://doi.org/10.1145/1806689.1806739
19. Nguyen, P.: Cryptanalysis of the Goldreich-Goldwasser-Halevi cryptosystem from crypto 1997. In: Wiener, M. (ed.) CRYPTO 1999. LNCS, vol. 1666, pp. 288–304. Springer, Heidelberg (1999). https://doi.org/10.1007/3-540-48405-1_18
20. Vaudenay, S. (ed.): EUROCRYPT 2006. LNCS, vol. 4004. Springer, Heidelberg (2006). https://doi.org/10.1007/11761679
21. Nguyen, P.Q., Valle, B.: The LLL Algorithm - Survey and Applications. In: Information Security and Cryptography. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-02295-1
22. Nguyen, P.Q., Vidick, T.: Sieve algorithms for the shortest vector problem are practical. J. Math. Cryptol. **2**(2), 181–207 (2008)
23. Plantard, T., Schneider, M.: Creating a challenge for ideal lattices. Cryptology ePrint Archive, Report 2013/039 (2013). https://ia.cr/2013/039
24. Pohst, M.E.: On the computation of lattice vectors of minimal length, successive minima and reduced bases with applications. SIGSAM Bull. **15**, 37–44 (1981). https://doi.org/10.1145/1089242.1089247
25. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. In: Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing. STOC 2005 (2005). https://doi.org/10.1145/1060590.1060603
26. Schneider, M., Gama, N.: Darmstadt SVP challenges (2010)
27. Schnorr, C.P., Hörner, H.H.: Attacking the Chor-rivest cryptosystem by improved lattice reduction. In: Guillou, L.C., Quisquater, J.-J. (eds.) EUROCRYPT 1995. LNCS, vol. 921, pp. 1–12. Springer, Heidelberg (1995). https://doi.org/10.1007/3-540-49264-X_1
28. Schnorr, C.P.: A hierarchy of polynomial time lattice basis reduction algorithms. Theor. Comput. Sci. **53**, 201–224 (1987)
29. Schnorr, C.P.: Lattice reduction by random sampling and birthday methods. In: Alt, H., Habib, M. (eds.) STACS 2003. LNCS, vol. 2607, pp. 145–156. Springer, Heidelberg (2003). https://doi.org/10.1007/3-540-36494-3_14

30. Schnorr, C.P.: Accelerated slide- and LLL-reduction. Electron. Colloquium Comput. Complex. TR11 (2011)
31. Schnorr, C.P., Euchner, M.: Lattice basis reduction: improved practical algorithms and solving subset sum problems. Math. Program. **66**, 181–199 (1994)
32. Yamaguchi, J., Yasuda, M.: Explicit formula for Gram-Schmidt vectors in LLL with deep insertions and its applications. In: Kaczorowski, J., Pieprzyk, J., Pomykała, J. (eds.) NuTMiC 2017. LNCS, vol. 10737, pp. 142–160. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-76620-1_9